Tencent 腾讯 / DataFunCon 2021

「解耦」方能「专注」——腾讯天穹SUPERSQL跨引擎计算揭秘

陈奕安



目录 CONTENT

01 背景目标

02 业界竞品

03 系统架构

04 元数据管理

05 跨源SQL算子下推

06 分布式计算引擎

07 跨DC查询优化

08 性能评测

01

背景目标

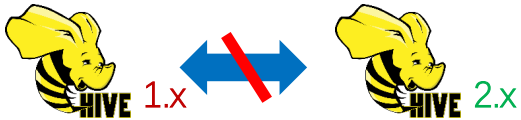


SuperSQL项目背景：问题与挑战

异构数据源

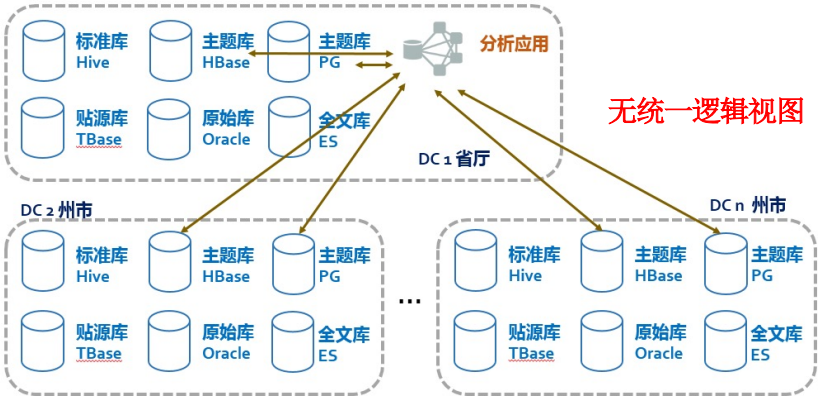


不同版本数据源



数据不互通，迁移困难（海量、业务、安全等）

现状: 应用而非平台层级的SQL拆分，人工数据搬迁与查询调优



无统一逻辑视图

数据源位于不同地域的多个DC/集群



带宽受限，端口/IP屏蔽

SuperSQL落地成果：支持跨业务平台（一）

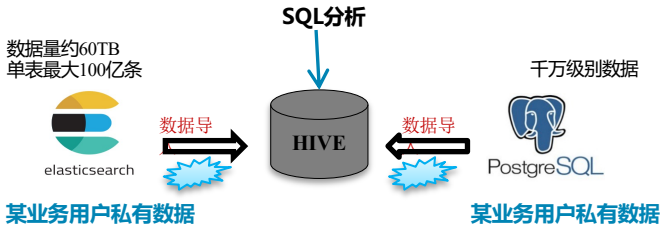
支持跨业务平台的数据融合，提升业务效率

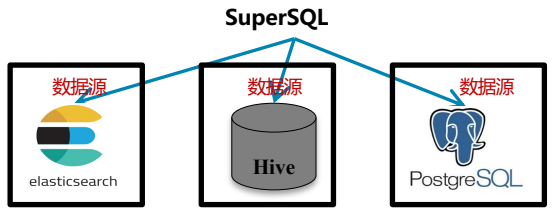
广告某业务场景(跨业务平台的数据分析)

- **业务需求**：联合查询/分析存储在**私有集群**的数据和**中心化数据仓库**中的业务数据，分析在**中心化数据仓库**
- **业务痛点**：
  - ✓ **数据导入中心化数据仓库费时费力**
  - ✓ **数据一致性无法保证**
  - ✓ **缺少统一的数据查询/分析入口**

SuperSQL效果

- **无需数据搬迁**：节省开发人力**5-10人**，节省导入时间**1~2h**
- **查询分析效率**：性能总体提升**3倍**以上
- **统一数据查询分析入口**：SuperSQL管理所有的数据源
- **整体一致性保证**：避免了ETL操作，一份数据，无需跨多个系统来验证备份数据的一致性





SuperSQL落地成果：支持跨业务平台（二）

支持跨业务平台的数据融合，提升业务效率

某业务场景(跨业务平台的数据分析)

- **业务需求**：联合查询/分析存储在**私有集群**的数据和**历史数据仓库**中的业务数据，分析在**业务平台**
- **业务痛点**：
  - ✓ **数据出库到业务平台流程复杂**
  - ✓ **数据任务脚本（Spark jar）维护困难**



SuperSQL效果

- **流程简单**：数据到业务平台只需SuperSQL，**数据无需中间落地**，节省导入时间约**1h**
- **维护简单**：数据任务由SQL完成，逻辑清晰



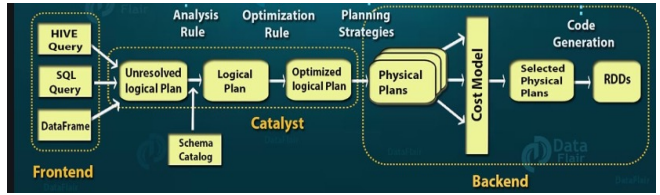
02
业界竞品



业界竞品分析：开源软件

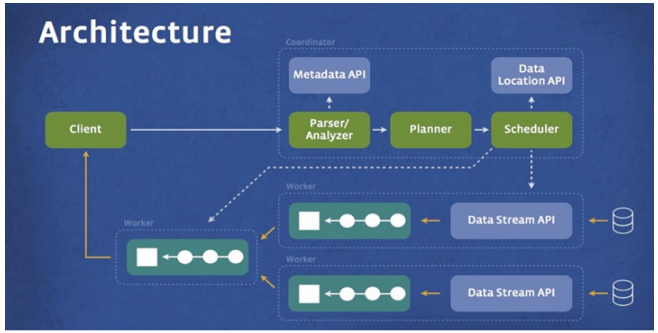
SparkSQL

- **简介**：DataFrame/DataSet编程抽象，分布式SQL查询
- **优势**：原生HDFS访问优化，DataSource API对接异构数据源，Catalyst SQL Parser/RBO，Stage失败重试
- **不足**：通用DAG而非专用SQL引擎，DataSource V1 仅支持简单Project/Filter下推数据源， DataSource V2尚待完善，CBO雏形且发展不明朗



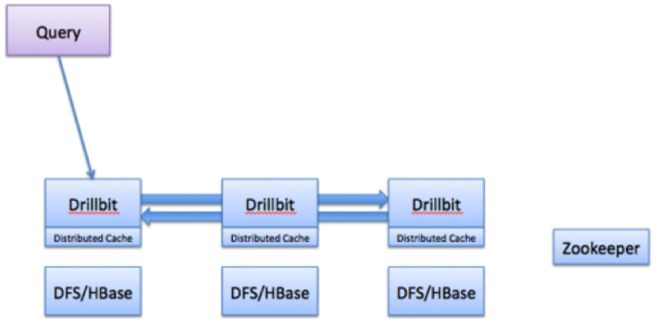
Presto

- **简介**：基于内存的并行计算，Facebook推出的分布式SQL交互式查询引擎，多个节点管道式执行
- **优势**：MPP模型，仅计算部分数据放置内存，多Connector支持异构源
- **不足**：不适合多个大表长时间的Join操作，全内存操作容易OOM，容错机制较差



Drill

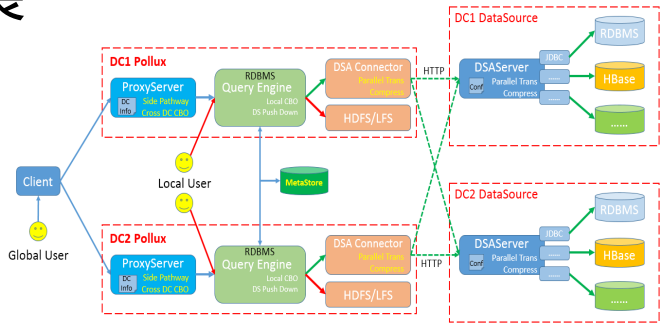
- **简介**：Google Dremel系统开源实现，MapR出品，低延迟交互式SQL分析
- **优势**：基于Calcite自研实现的分布式SQL执行引擎，MPP模型，P2P架构
- **不足**：多源对接功能基本源于社区Calcite无扩展，过于聚焦HDFS加速，稳定性和社区生态无法匹及Spark/Presto



业界竞品分析：开源扩展

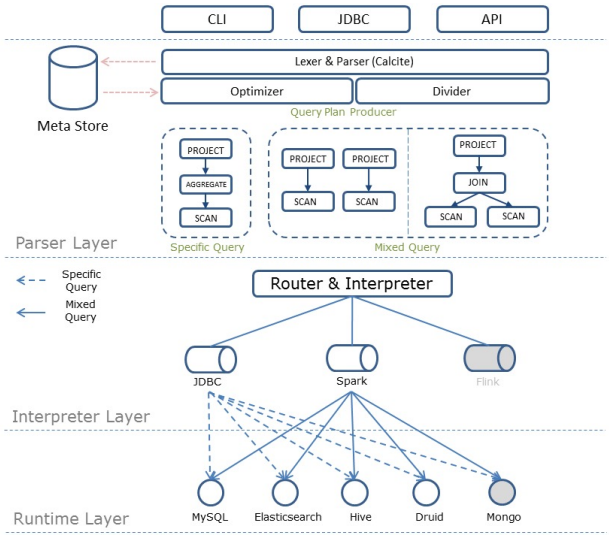
Pollux (未开源)

- **简介**：跨DC多数据源的协同计算，基于开源SparkSQL深度定制扩展
- **优势**：复杂算子下推，跨DC CBO，DSA传输加速，Spark任务日志
- **不足**：可扩展性不太好，与Spark代码强耦合(侵入式修改)，无Join下推支持，CBO依赖于Spark社区尚未成熟的方案



Quicksql (开源)

- **简介**：基于Calcite扩展实现，具备多数据源的接入能力，解析转化用户提交的查询语句，最终提交到Jdbc/Spark/Flink执行
- **优势**：整体的架构方案做到了轻量级，提供单DC CBO与外部计算引擎对接
- **不足**：核心功能基于开源Calcite，未解决社区版本的一系列问题，例如多Jdbc源SQL仅单源能下推、复杂查询无法生成可执行计划或CBO耗时过长等；开源版本状态几乎无法商用，元数据机制、Spark/Flink对接代码均仅达到Demo级别；不支持跨DC CBO与查询执行

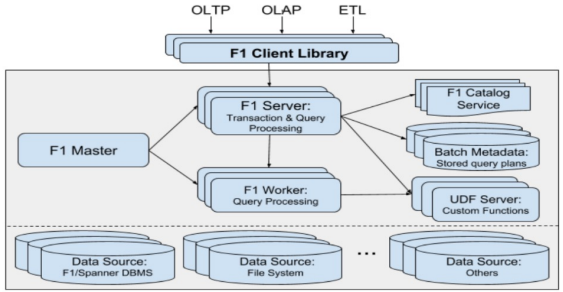


业界竞品分析：私有系统

F1-Query (未开源)

- **简介**：Google内部异构数据源查询引擎，支持对各种不同文件格式和存储系统(Bigtable, Spanner, Google Spreadsheets) 中的数据进行跨DC的联合查询
- **优势**：各模块(如Metadata、UDF统一管理)的抽象设计具有较为通用的参考价值
- **不足**：仅用于Google内部OLTP场景，私有系统参考文献较少

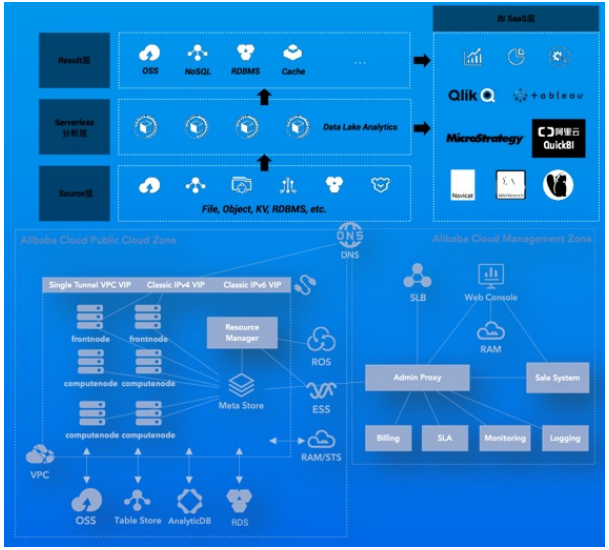
Google



Data Lake Analytics (未开源)

- **简介**：阿里自研的数据湖产品，为用户提供跨源的联合分析能力，内部自研组件 + Presto扩展
- **优势**：支持多路数据源接入，提供多样化、异构数据源关联分析能力，融合MPP和DAG技术，具备横向分析扩展、向量化执行、算子流水线优化等能力
- **不足**：无法灵活地管理数据源间的层级关系，资源管理、任务调度执行等重复“造轮子”，技术架构针对特定场景的定制化程度高，通用适配性一般

Alibaba.com



业界竞品分析：华为河图（HetuEngine）



数据虚拟化引擎：像使用“数据库”一样使用“大数据”，Pollux衍生扩展

HetuEngine引擎可对底层各数据中心的RDBMS、HDFS、ElasticSearch、Redis、In-Memory DB、Kafka等进行统一的数据管理和治理

- 核心能力
  - “一个目录”：数据全域共享、打破数据孤岛、支持1000+数据源
  - “一个接口”：统一SQL接口、降低开发成本，支持5000+节点
  - “一份数据”让数据零搬迁、提升时效性、基于CarbonData
  - “统一安全”完整保护数据、保障安全开放、细粒度动态授权
- 当前为私有系统，公开资料甚少，计划2020年6月内核开源，尚无性能数据
  - “开发者可以基于开源代码进行定制，包括数据源扩展、SQL执行策略等”
  - “确保合作伙伴们能够免厂商锁定”
  - “复用现有的生态、工具和技能，提升开发效率2到10倍”

北向应用接口 +

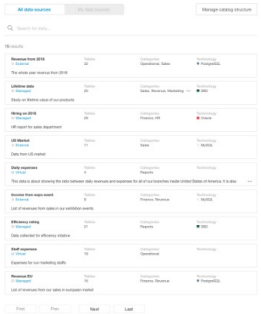
南向数据接口 +

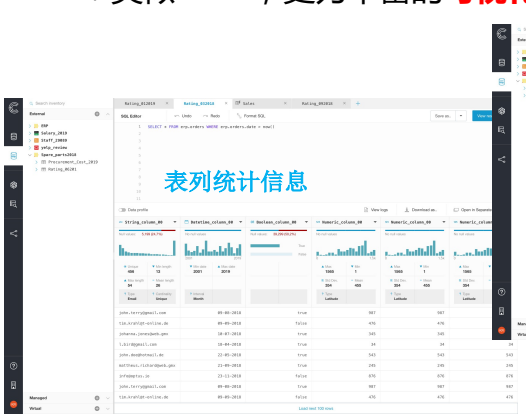
内核引擎

业界竞品分析：Contiamo Workbench

- 德国“Start-up”大数据公司，成立于2013年，CTO是Calcite Committer
- Calcite扩展，使用Spark作为外部计算引擎，技术方案基本与SuperSQL一致
- UI：类似IDEX，更为丰富的**可视化**功能组件，提升用户体验

数据源列表







支持数据源类型（混合云）



SuperSQL产品化指引

SuperSQL vs. 商业竞品：更丰富完善的功能

维度	DLA (阿里)	Hetu (华为)	Contiamo	SuperSQL
数据源	NoSQL：TableStore、OSS、MongoDB、Redis RDBMS：MySQL、PostgreSQL、SQL Server、Oracle	Pollux： Hive、MPPDB、Oracle、MySQL Hetu：ES、Redis、Kafka	RDBMS：Oracle、MySQL、PG、SQL Server、Hana、DB2 Cloud：S3、Azure	**RDBMS：MySQL、PostgreSQL、Oracle、ClickHouse ...** **NoSQL：Hive、Phoenix、ES、Kylin、Hermes ...**
支持SQL	MySQL 5.7绝大部分查询语法	Spark-SQL文法	SQL-2003标准	**SQL-2003标准**
元数据	内部映射	Spark Catalog	未知	**独立模块，Hive MetaStore持久化**
查询优化	① 支持OSS Select单文件的计算下推优化 ② 针对Table Store SDK进行列式压缩调用返回优化	① 计算下推优化：Aggregate/OrderBy/Limit/复杂表达式下推 ② 跨DC查询的CBO优化	基于规则的优化器(RBO)，支持各类算子下推	**① 基于规则的优化器(RBO)，支持各类算子下推** **② 多阶段Planner规则切分** **③ 单DC内代价优化器(CBO)** **④ 跨DC的代价优化器(CBO)**
计算引擎	自研（/Presto）	Spark(/Presto...)	Spark	**Hive/Spark/Livy/...智能选择**

技术断裂点/趋势：

跨源：类型多样性，语法兼容性，系统稳定性

跨DC：CBO因子，子查询分布式协同执行

跨引擎：耦合度，性能，升级/替换灵活性，负载/场景适应性

平台、成品、实战

03
系统架构



SuperSQL内核：Apache Calcite数据管理框架

- 工业级的SQL Parser与Validator
- 关系代数变换：SQL -> SqlNode -> RelNode -> SQL
- RBO/CBO框架：RelOptRule + HepPlanner & VolcanoPlanner
- 元数据接口定义：Schema/Table/…
- 多类数据源连接器 (Adapter)
- JDBC Client/Server (Avatica)
- 单机单线程Enumerable执行引擎 (Linq4J)

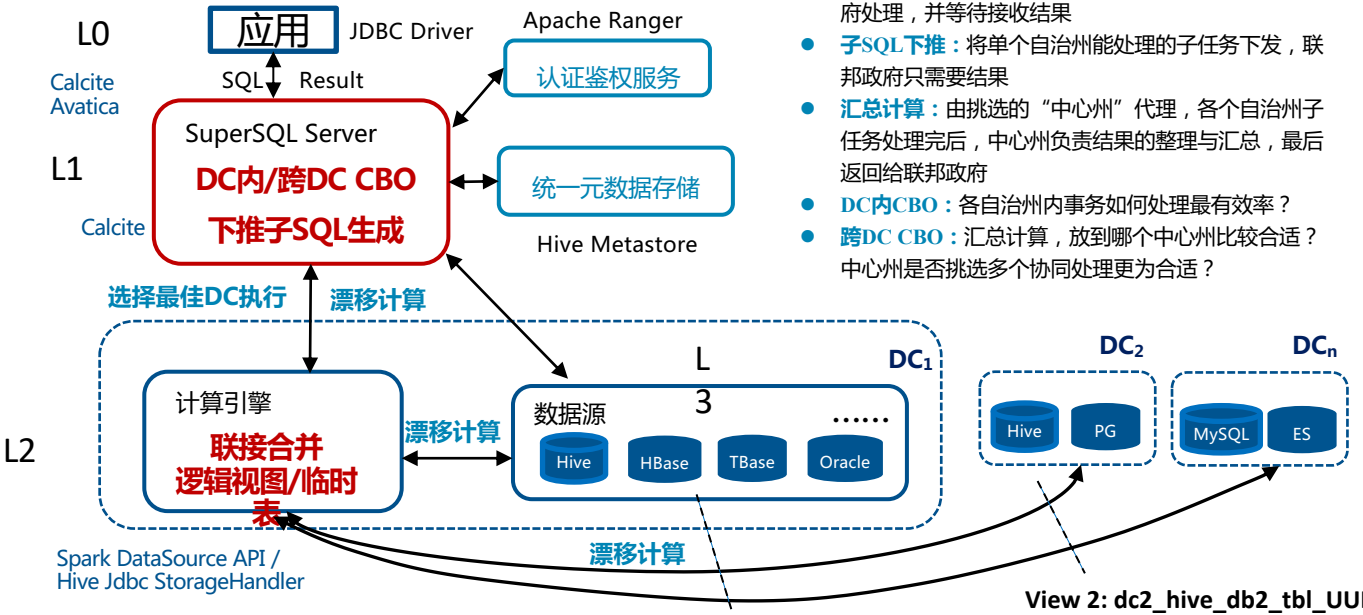


Calcite局限性：框架而非平台，原料而非成品，注重功能示例而非实战使用

SuperSQL: Calcite ++

SuperSQL整体架构：漂移计算

Query:
(SELECT t1.a, count(t1.b) as cnt FROM dc1.tbase.db1.tbl AS t1
WHERE t1.c > 100
GROUP BY t1.a ORDER BY t1.a LIMIT 10)
UNION ALL
(SELECT t2.a, count(t2.b) as cnt FROM dc2.hive.db2.tbl t2
GROUP BY t2.a LIMIT 20)



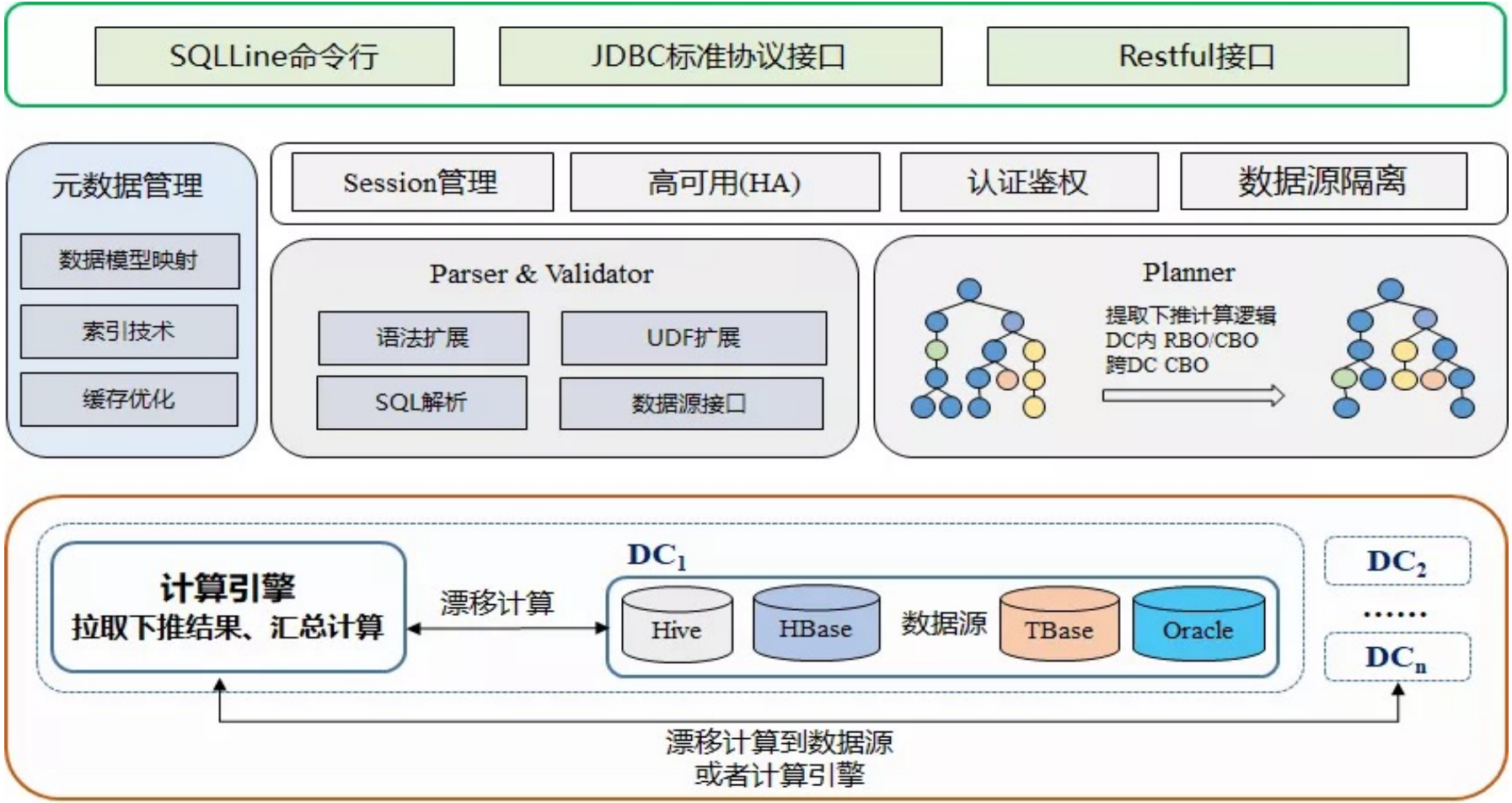
角色类比

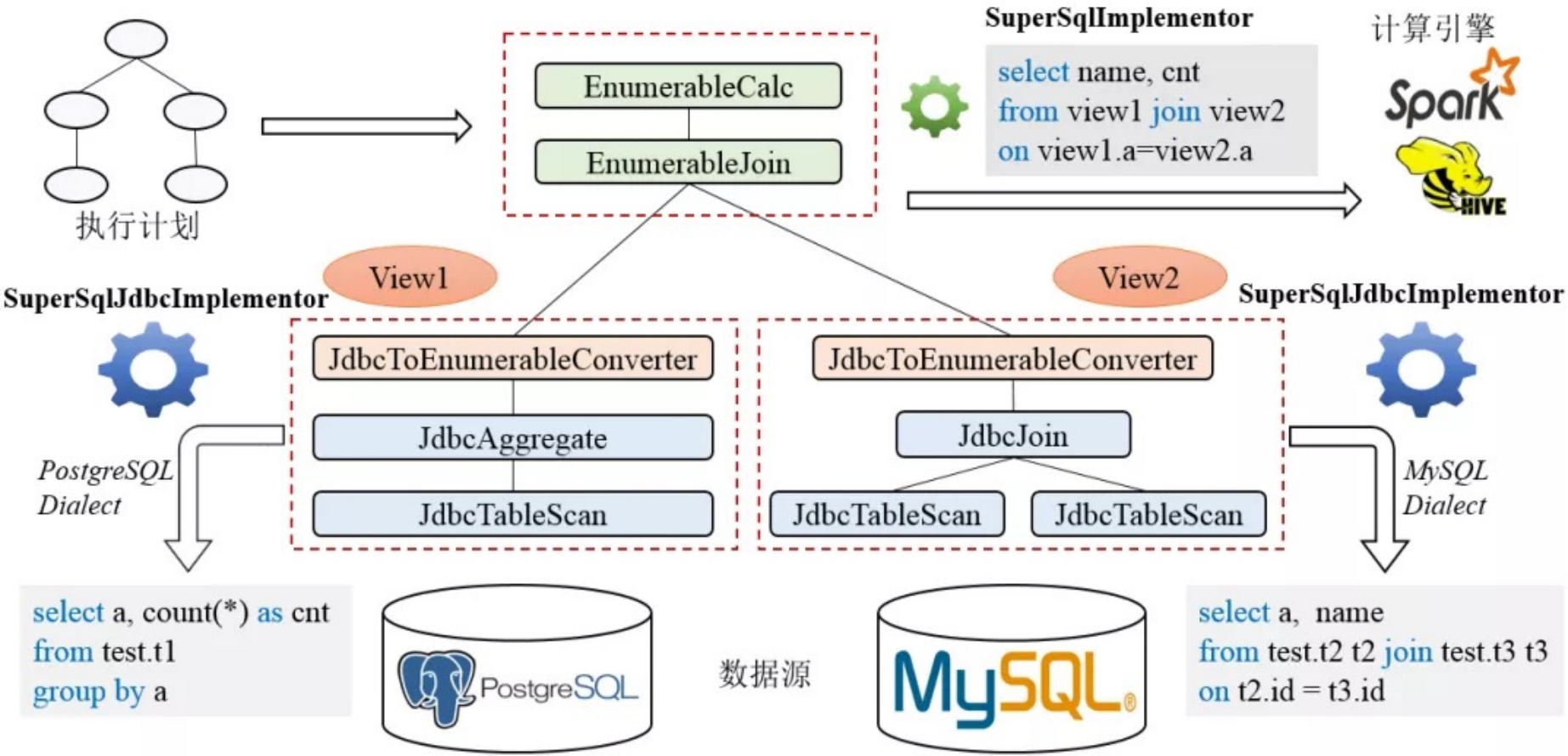
- **SuperSQL**：联邦政府
- **DC(数据中心)**：各个“自治州”政府
- **用户SQL**：涉及多个自治州的任务，统一提交联邦政府处理，并等待接收结果
- **子SQL下推**：将单个自治州能处理的子任务下发，联邦政府只需要结果
- **汇总计算**：由挑选的“中心州”代理，各个自治州子任务处理完后，中心州负责结果的整理与汇总，最后返回给联邦政府
- **DC内CBO**：各自治州内事务如何处理最有效率？
- **跨DC CBO**：汇总计算，放到哪个中心州比较合适？中心州是否挑选多个协同处理更为合适？

计算引擎改写SQL:
(SELECT * FROM dc1_pg_db1_tbl_UUID)
UNION ALL
(SELECT * FROM dc2_hive_db2_tbl_UUID)

View 1: dc1_pg_db1_tbl_UUID
SELECT t1.a, count(t1.b) as cnt FROM dc1.tbase.db1.tbl AS t1
WHERE t1.c > 100 GROUP BY t1.a ORDER BY t1.a
LIMIT 10

View 2: dc2_hive_db2_tbl_UUID
SELECT t2.a, count(t2.b) as cnt
FROM dc2.hive.db2.tbl t2
GROUP BY t2.a
LIMIT 20


SQLLine命令行
JDBC标准协议接口
Restful接口
元数据管理
数据模型映射
索引技术
缓存优化
Session管理
高可用(HA)
认证鉴权
数据源隔离
Parser & Validator
语法扩展
UDF扩展
SQL解析
数据源接口
Planner
提取下推计算逻辑
DC内 RBO/CBO
跨DC CBO
计算引擎
拉取下推结果、汇总计算
漂移计算
DC1
Hive
HBase
数据源
TBase
Oracle
DC2
.....
DCn
漂移计算到数据源
或者计算引擎


执行计划
EnumerableCalc
EnumerableJoin
SuperSqlImplementor
select name, cnt
from view1 join view2
on view1.a=view2.a
计算引擎
Spark
HIVE
View1
View2
SuperSqlJdbcImplementor
SuperSqlJdbcImplementor
JdbcToEnumerableConverter
JdbcAggregate
JdbcTableScan
JdbcToEnumerableConverter
JdbcJoin
JdbcTableScan
JdbcTableScan
PostgreSQL
Dialect
MySQL
Dialect
select a, count(*) as cnt
from test.t1
group by a
PostgreSQL
数据源
MySQL
select a, name
from test.t2 t2 join test.t3 t3
on t2.id = t3.id

04

元数据管理

元数据 = 数据源信息 + CBO统计信息



数据源元数据模型

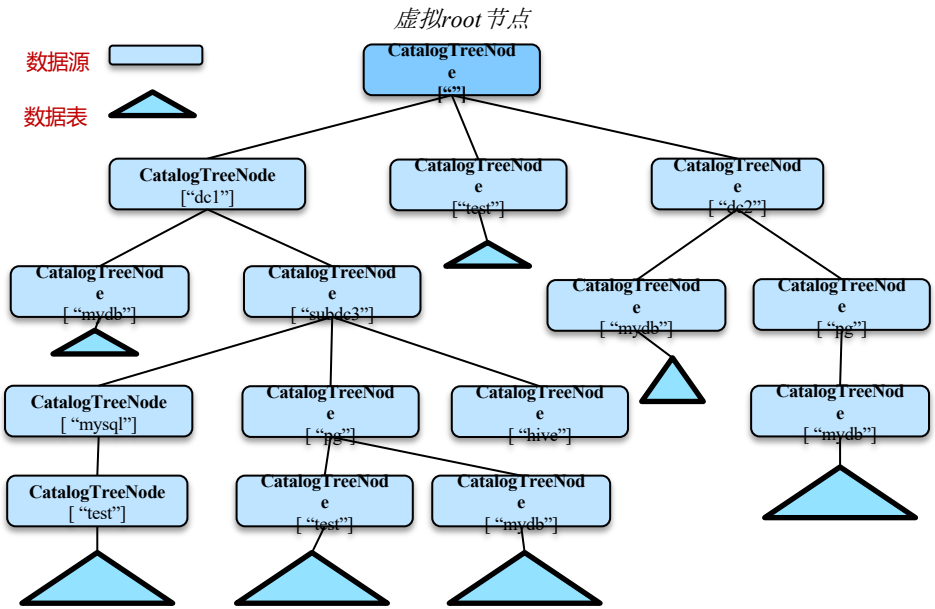
元数据模型设计

- **树结构**：特殊的Trie，每个节点对应的子树形成了一个子命名空间
- **树节点**：CatalogTreeNode主要内容有父节点parent、子节点列表children、节点路径path、名称name（path中最后一个元素），以及Map类型的conf配置信息

```
select *
from test.mytable
```

⇩

```
select *
from
dc1.subdc3.pg.test.mytable
```

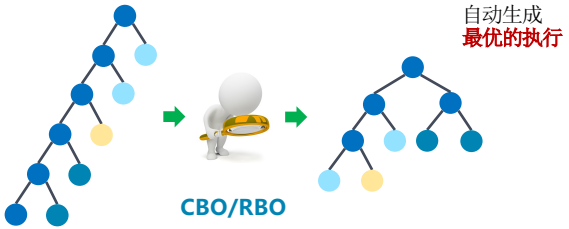


SuperSQL元数据管理I：层级数据源表征

- 通用性：解决传统的DB/Table模型面临的同库同表等问题
- 灵活性：数据源定义需要非常灵活，上下层级关系可任意定义，物理世界实体的逻辑抽象（主数据中心 -> 子数据中心 -> 机架 -> 集群 -> 数据库系统）
- 轻量级：最大化地复用现有开源组件，避免引入额外维护开销

SuperSQL元数据管理II：CBO统计信息采集

- CBO需要估算SQL执行计划中每个算子的代价，包括CPU、IO、Network等
- 一个物理计划的代价，就是其树型结构上所有节点（算子）的代价之和
- 在无数候选计划中，CBO挑选代价最小的计划，作为最优计划来执行
- 估算某个SQL算子（如Filter、Join、Sort）的代价，需要首先预估其输出和处理的行数/字节数，其基础为每个底层Scan算子的预估值，通过采集对应数据源表的统计信息（CBO Stats）来计算
- 数据源表的统计信息分为表级和列级两类
  - 表级：行数，字节数
  - 列级：



用户只关心业务逻辑
不懂SQL优化，
不关心SQL不同写法

数值型

Numeric/Date/Timestamp type
- ✓ Distinct count
- ✓ Selectivity
- ✓ Max
- ✓ Min
- ✓ Null count
- ✓ Average length (fixed length)
- ✓ Max length (fixed length)

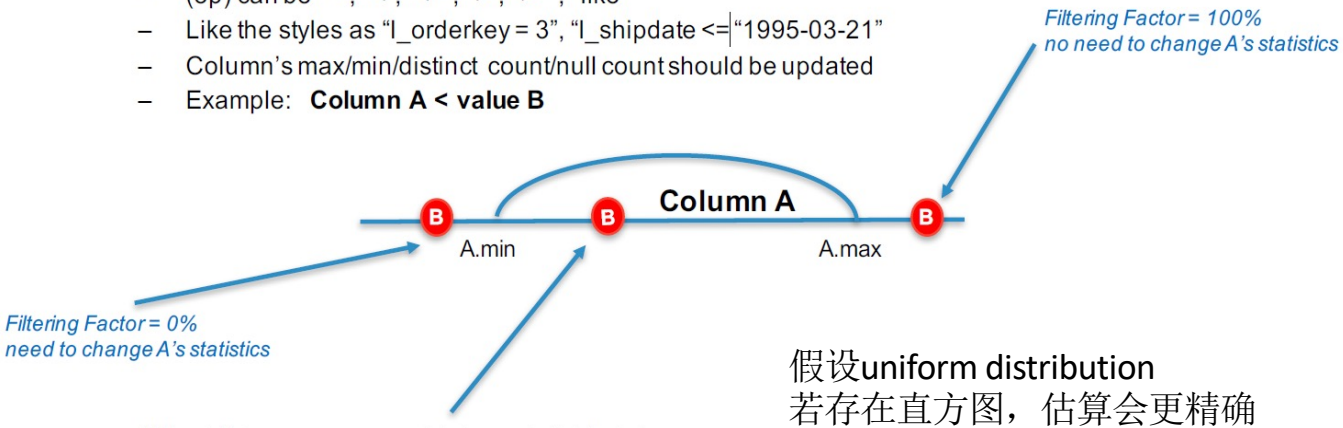
字符串型

String/Binary type
- ✓ Distinct count
- ✓ Null count
- ✓ Average length
- ✓ Max length

社区Calcite有Statistics接口，但无统计信息采集和使用的逻辑实现

统计信息使用样例：Filter输出估算

- ***Column A (op) literal B***
  - (op) can be “=“, “<”, “<=”, “>”, “>=”, “like”
  - Like the styles as “l_orderkey = 3”, “l_shipdate <=“1995-03-21”
  - Column’s max/min/distinct count/null count should be updated
  - Example: **Column A < value B**



假设uniform distribution
若存在直方图，估算会更精确

先估算输出rowCount，然后根据
各列的平均长度计算sizeInBytes

统计信息持久化：Hive MetaStore

问题：同一数据源的所有表的Cbo Stats会被同时读写，效率低下，造成Hive MetaStore性能压力

- 当前SuperSQL对接Hive Metastore方式
  - 专用一个名为“supersql”的数据库
  - 每个注册到SuperSQL 的数据源（DataSource），对应该数据库中的一张虚拟表
  - DataSource属性（config）存入Hive Metastore的TABLE_PARAMS表中
- CBO Stats使用类似方式存入Hive Metastore
  - 参数名：*supersql.cbo.tableStats.srcTableName.segment.X*
  - 参数值：TableStat（含ColumnStats）对象序列化为字节数组后，base64编码
  - 因参数值有最大长度限制，分段（segment）存储（X = 1, 2, …）
  - 特殊参数 “*supersql.cbo.tableStats.srcTableName.segment.-1*” 存储segment总数

srcTableName是数据源中某张数据表的名称

Supersql DataSource 对应的Hive虚拟表 ID

Stats存储格式样例

TBL_ID	PARAM_KEY	PARAM_VALUE
1	supersql.cbo.tableStats.table1.segment.0	XXXXXXXX…
1	supersql.cbo.tableStats.table1.segment.1	XXXXXXXX…
1	supersql.cbo.tableStats.table1.segment.2	XXXXXXXX…
1	supersql.cbo.tableStats.table2.segment.-1	2
2	supersql.cbo.tableStats.table3.segment.0	XXXXXXXX…

统计信息采集：Analyze 命令

```
ANALYZE TABLE datasourceName.tableName
COMPUTE STATISTICS
[FOR COLUMNS columName1, columName2, …]
[WITH SAMPLE SIZE sampleNum]
[WITH SAMPLE TIMES sampleTimes]
[WITH INCREMENT condition | RECOMPUTE]
```

手动执行，
非系统自动触发，
可配置定期脚本

- 扩展Calcite Parser实现，单一命令同时**采集表级 + 列级统计信息**
  - 表级Stats必采：简单、高效
  - 若不包含FOR COLUMNS子句，默认采集所有列的Stats；若包含则仅采集指定名称的列，至少需指定一列
- WITH SAMPLE SIZE指定采样的行数
  - 默认值设置为 N = min(tableRowCount, 1000w)
  - **全表随机采样**，如果数据源表包含分区且该信息可以通过Jdbc接口获取，则每个分区均随机采样1/N
- WITH SAMPLE TIMES指定采样的次数，多次采样可以增加随机概率
  - 默认值为1，多次采样的结果**增量合并**
  - **累积采样效果**，提升Stats精度，从而潜在提升CBO的精度
- 同一数据表先后执行多次Analyze命令，默认更新模式是RECOMPUTE
  - RECOMPUTE子句（可选）指定本次命令指定为全量覆盖，即删除不合并之前采集的Stats，适用于表数据变化较大或历史Stats采集明显不准确的情况
  - WITH INCREMENT子句（可选）指定一个条件，如dp = ‘2019-08-01’ and cp > 100，命令执行时仅随机采样满足该条件的数据，且采样结果增量合并入当前Stats，适用于新增或删除表分区的情况

列统计信息采样与合并逻辑

列Stats	采集函数	合并逻辑（前后两次采样）
ndv	COUNT(DISTINCT col)	按采样行数比例求均值，ratio = N1 / (N 1 + N2 ），ndv = ndv1 * ratio + ndv2 * (1 - ratio)
nullCount	SUM(CASE WHEN col IS NULL THEN 1 ELSE 0 END)	按采样行数比例求均值
max	MAX(col)	MAX(max1, max2)
min	MIN(col)	MIN(min1, min2)
maxLen	MAX(LENGTH(col))	MAX(maxLen1, maxLen2)
avgLen	AVG(LENGTH(col))	(avgLen1 * rowCount1 + avgLen2 * rowCount2) / (rowCount1 + rowCount2)
selectivity	SUM(CASE WHEN col THEN 1 ELSE 0 END) / COUNT(1)	(selectivity1 * rowCount1 + selectivity2 * rowCount2) / (rowCount1 + rowCount2)
histogram	DISTINCT col，SuperSQL端通过HyperLogLog（HLL） [4] 来计算和表示，每个直方图bucket对应一个HLL数据结构	1. SuperSQL采用**近等高**直方图，即不强制要求所有bucket中的值数必须相等 2. 后续采样沿用前次采样的bucket边界， 同边界的两个bucket对应的HLL merge 3. 最大最小bucket的高度差，若超过系统定义阀值则需部分调整（合并前后bucket），若调整失败则全量重构直方图

05

跨源SQL算子下推



SuperSQL：一站式高性能大数据SQL中间件

- 跨源：支持访问**不同类型/版本**的数据源 – Hive // SparkSQL / PG / TBase / MySQL / Oracle / H2 / HBase (Phoenix Query Server) / ElasticSearch (sql4es) / Kylin / ClickHouse / Druid / PrestoDB / PrestoSQL / *SuperSQL*
- 跨引擎：支持外接多类**分布式**计算（SQL执行）引擎 – Hive（2.3.x）/ Spark（Livy、Kyuubi）/ Flink / Presto
- 跨DC：支持**跨数据中心/集群**的SQL优化与执行 – RBO/CBO

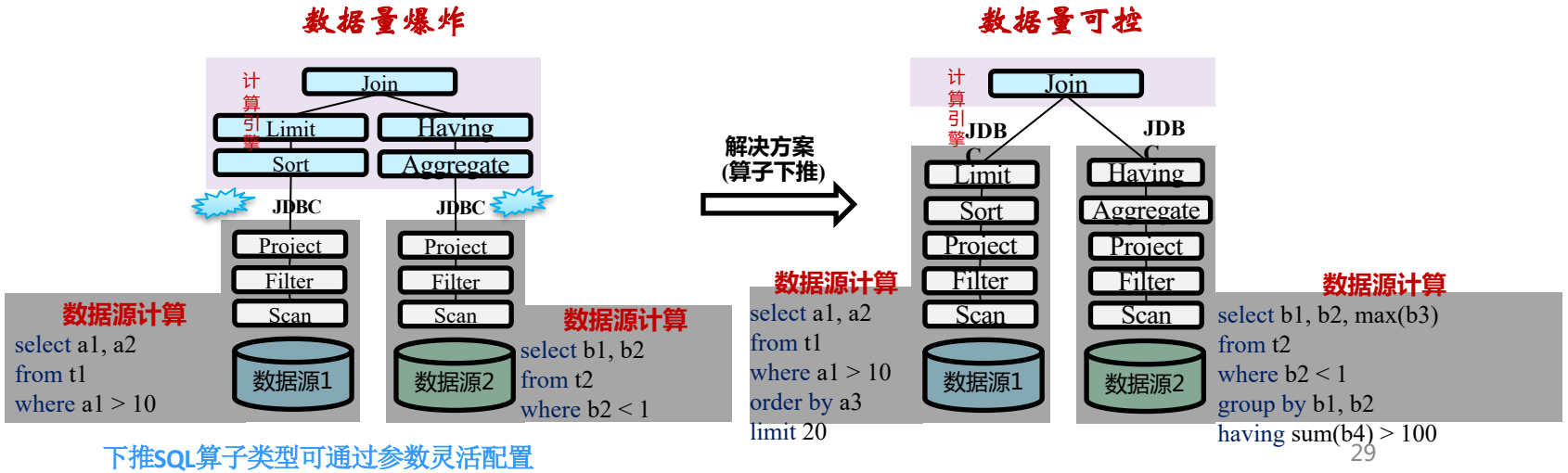


计算下推：支持常用复杂SQL算子

- **网络开销大性能低**：计算引擎通过JDBC单线程拉取大量数据，网络IO大且速度慢
- **计算引擎负载过重**：计算逻辑都在汇总的计算引擎完成，其所在集群负载压力大
- **资源使用不合理**：数据源的计算资源处于闲置状态，未能充分利用

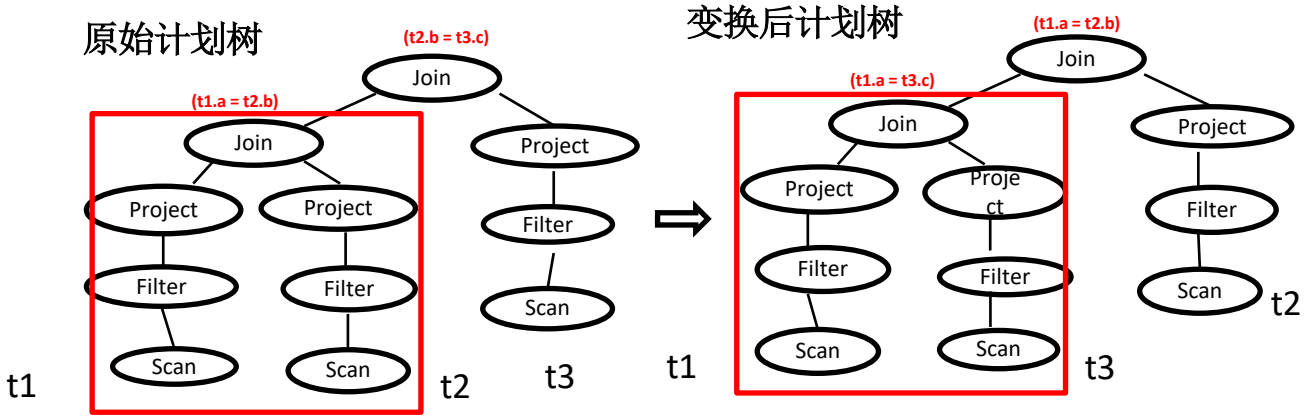
扩展60+下推规则实现

SQL算子类型	说明
聚合函数 (Aggregate)	包括count、sum、average、max、min等
Top N (Limit/Offset)	不同数据源语法不同，如Oracle为rownum < N
排序 (OrderBy)	Order-By不下推对应Limit无法下推
Union/Intersect/Minus	去重可以减少数据量
系统标量函数(Scalar Function)	length、nvl、datetime、power等
Join	CBO估算Join的结果是否会膨胀，权衡传输与计算开销；Join的重排序
数据源专有函数	下推到数据源执行，如tHive UDF/UDAF



下推SQL算子类型可通过参数灵活配置

Join下推示例



t1和t2来自不同数据源，无法下推
这里假设Join的执行次序一定是从左到右。

假设t1和t3来自同一数据源。
Join重排序过程中，还要考虑Join条件的自动传递，当前只考虑Equality条件。

Aggregate下推Union All（1/2）

仅考虑最常用的count/sum/avg/max/min聚合函数，不支持count distinct

t1和t2来自不同DC的数据源

```
select count(*)
from (select id from t1
union all
select id from t2) t
```



```
select sum(cnt)
from (select count(id) as cnt from t1
union all
select count(id) as cnt from t2) t
```

```
select max(id)
from (select id from t1
union all
select id from t2) t
```



```
select max(id)
from (select max(id) as id from t1
union all
select max(id) as id from t2) t
```

```
select avg(num)
from (select num from t1
union all
select num from t2) t
```



```
select sum(s)/sum(c)
from (select sum(num) as s,  count(num) as c from t1
union all
select sum(num) as s, count(num) as c from t2) t
```

不能是Union，因为涉及去重时无法确保语义正确

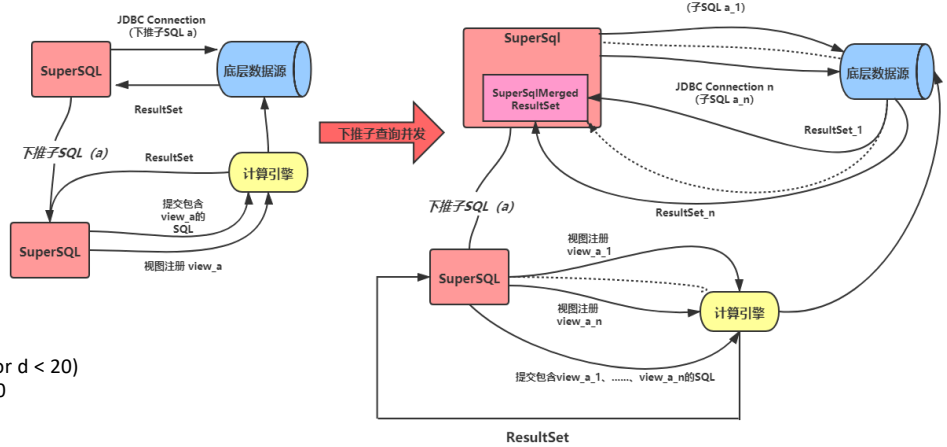
好处：能大量减少数据传输

下推SQL优化：并发子查询切分

动机：当下推某一数据源的SQL返回数据量很大时（如千万条记录，可通过CBO估算），单个JDBC连接获取结果耗时长

适用场景：基于统计信息，发现数据源表有合适的分区（如Hive）或Unique索引（如TBase）

当前局限：只支持简单Select-Where查询，不支持带Agg./OrderBy/ Limit/Join的查询（语义复杂，无法确保拆分子查询结果合并的正确性）




```
select a, b, c
from t1
where a > 10
or d < 20
```

```
select a, b, c
from t1
where (a > 10 or d < 20)
and parCol < 10
```

```
select a, b, c
from t1
where (a > 10 or d < 20)
and parCol >= 10 and parCol < 20
```

```
select a, b, c
from t1
where (a > 10 or d < 20)
and parCol >= 20 and parCol < 30
```

原理：

1. 根据分区、索引或其它列统计信息，为下推数据源表挑选一个“最好的切分列”parCol
2. 基于parCol的最大最小值生成N个disjoint的分区条件（N可以固定也可以动态根据数据分布确定）
2. 将分区条件拼接到原始下推SQL的Where条件中，生成N个对应的子查询
3. 并发多线程执行N个子查询，合并ResultSet

06

分布式计算引擎

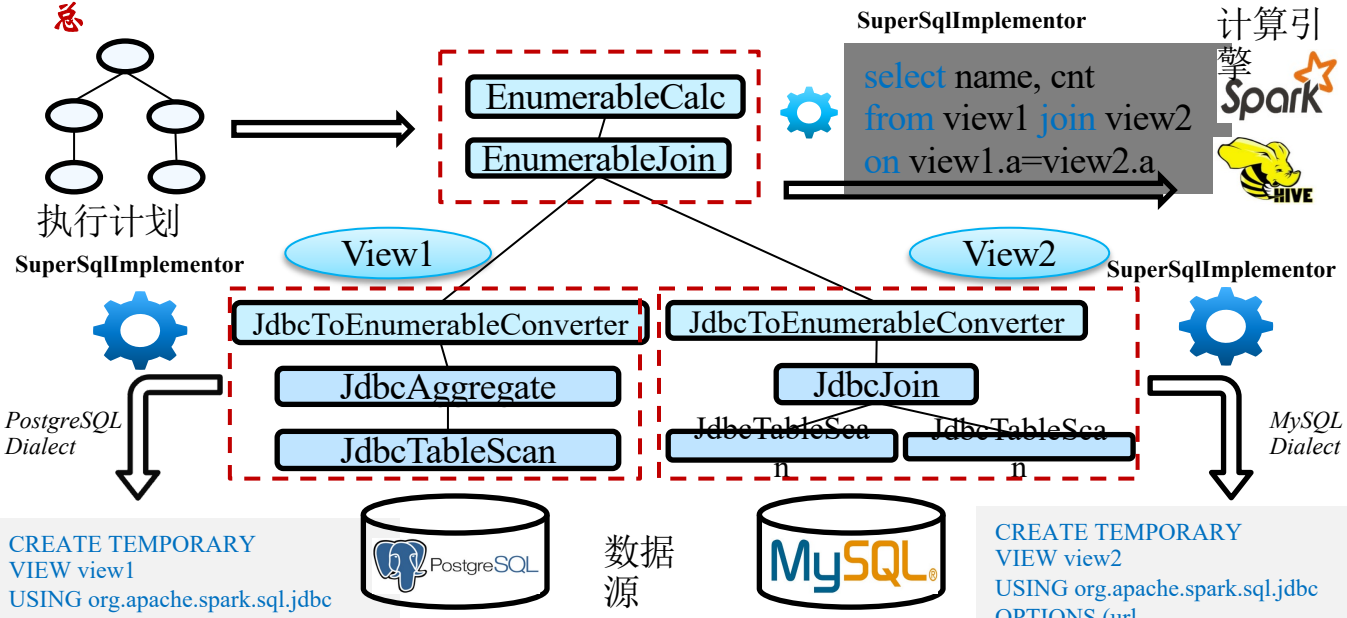


SuperSQL分布式计算引擎需求

- **快速构建**：不重复“造轮子”，尽量复用通用开源软件
- **轻量级解耦**：不强依赖特定框架，不做或少做侵入性修改
- **场景适配**：根据SQL特征，智能挑选主流执行引擎

计算引擎处理流程：数据源下推子计划 ->临时表映射

SuperSQL CBO-> 通知计算引擎 (二次优化) -> 数据源执行 -> 计算引擎汇总




```
CREATE TEMPORARY
VIEW view1
USING org.apache.spark.sql.jdbc
OPTIONS (url
'jdbc:postgresql://host1:port1/db1',
  dbtable '(select a, count(*) as cnt
from t1 group by a) t',
user "******", password "******");
```

SuperSQL外接执行引擎要求：

1. 提供JDBC Client/Server
2. 能够通过JDBC/HDFS/定制Connector连接到各类SuperSQL数据源
3. 支持动态增加/删除数据源或数据源表

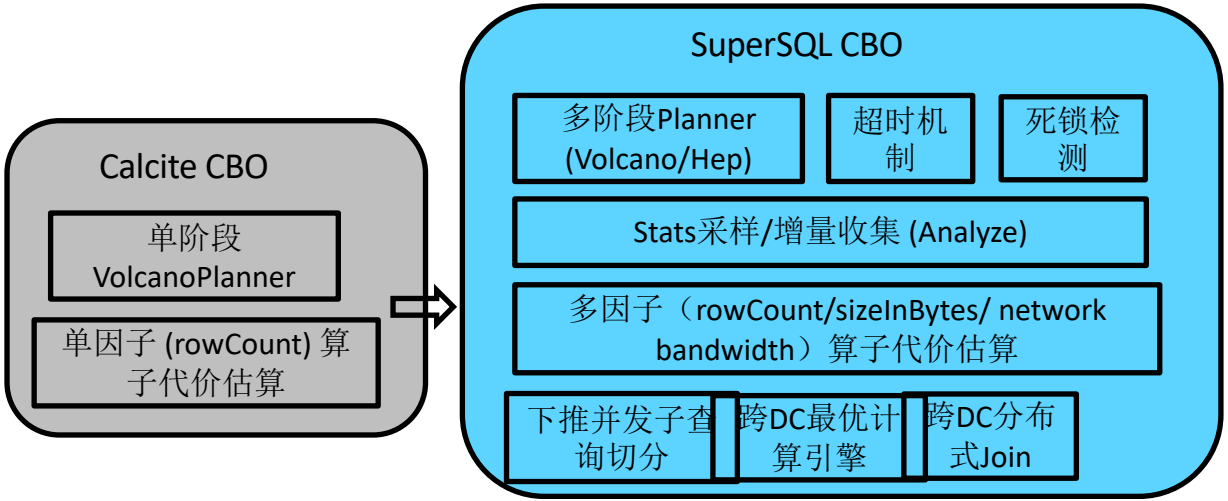
```
CREATE TEMPORARY
VIEW view2
USING org.apache.spark.sql.jdbc
OPTIONS (url
"jdbc:mysql://host2:port2/db2",
  dbtable '(select a,  name from t2
join t3 on t2.id = t3.id) t',
user "******", password "******");
```

07

跨DC查询优化



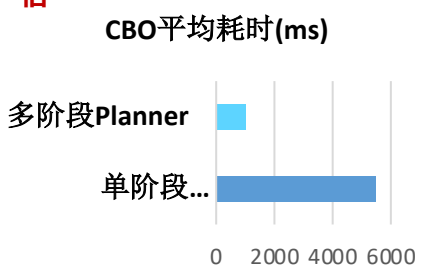
SuperSQL CBO概览



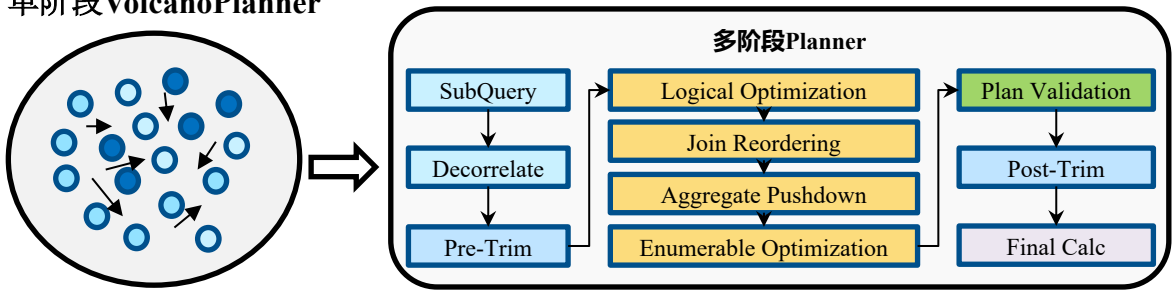
DC内CBO：多阶段Planner

- **超时机制**：加入软硬超时阀值，禁用计算开销大的规则或强制终止，可选次优执行计划
- **多阶段Planner**：根据变化目标，将规则集合拆分成多个串行阶段，每个阶段对应独立的规则子集
- **验证逻辑**：检测相互冲突的规则，避免形成“死锁”似的循环规则匹配

多阶段Planner CBO比单阶段约快5倍



单阶段VolcanoPlanner



单一队列，重复冗余匹配

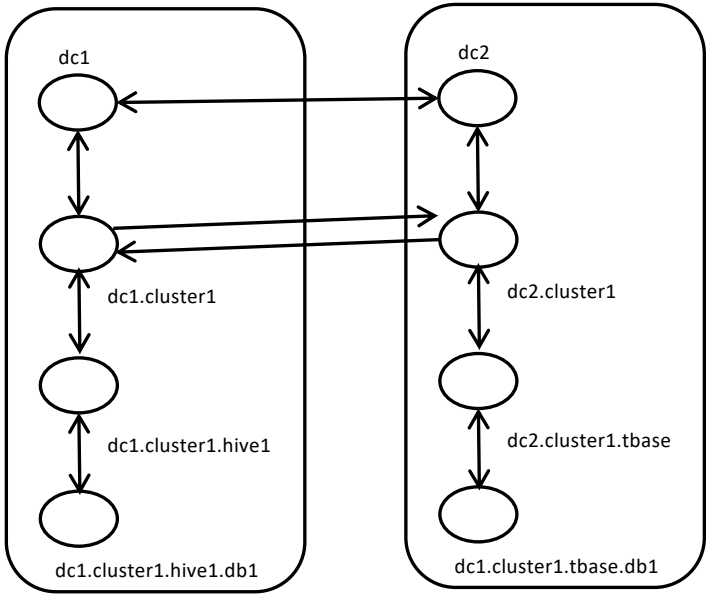
复杂SQL CBO卡住数十分钟，经常无法生成可执行

跨DC CBO：数据源间网络带宽

在创建/修改数据源的SQL命令中，通过bandwidth可选参数，指定该数据源到其它数据源之间的静态网络带宽

```
CREATE/ATLER DATASOURCE dc1.cluster1.hive1.db1
with (datasource=jdbc, catalog=tpcds_1g, …,
bandwidth=`[dc2:1g, dc3.cluster2:100m]`)
```

100m bit/s



基于所有DataSource相关属性值，构建**数据源带宽图**

- 每个合法的数据源名前缀，均对应一个节点
- 所有节点到其它任何节点均有边，边的权重代表数据从节点A发送到节点B的**单向**网络带宽
- 如果A到B的带宽已指定，但B到A的带宽未指定，则认为带宽对称
- 如果A到B的带宽未指定，A到B的带宽定义为A的父节点到B的带宽
- 如果A到B的双向带宽均未指定，则采用默认值10gb/s（万兆）
- 带宽图系统启动时加载内存，基于CREATE/ALTER DATASOURCE命令**实时更新**
- 静态带宽配置，对CBO起指导性作用（*scale but not exact value matters*），暂未考虑**动态带宽监控**

跨DC CBO：计算引擎到数据源的网络带宽

```
supersql.spark.thriftserver.url.dc1=jdbc:hive2://host1:port1/supersql
supersql.hive.thriftserver.url.dc2.cluster1=jdbc:hive2://host2:port2/supersql
```

- 多个计算引擎（Spark/Hive ThriftServer2），通过特定的带DataSource前缀名字符串的参数来指定和区分，参数名语义上表示计算引擎所部署的DC/集群
- 计算引擎A到数据源B的带宽（写入），定义为带宽图中A的DS前缀名对应节点到数据源名对应节点间的带宽，B到A的带宽同理定义（读取）

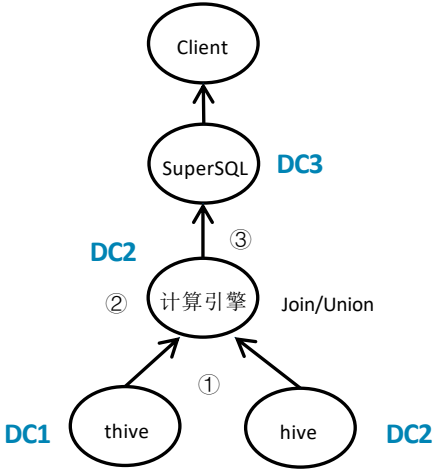
```
supersql.webserver.loc=dc1.cluster3
```

- 除外接计算引擎执行外，应用提交的SQL，也可以在SuperSQL Server所在的JVM进程使用Calcite原生引擎执行，上面的参数定义了SuperSQL Server对应的部署位置
- SuperSQL Server也可以看做一个计算引擎，它与数据源间的带宽同上获得

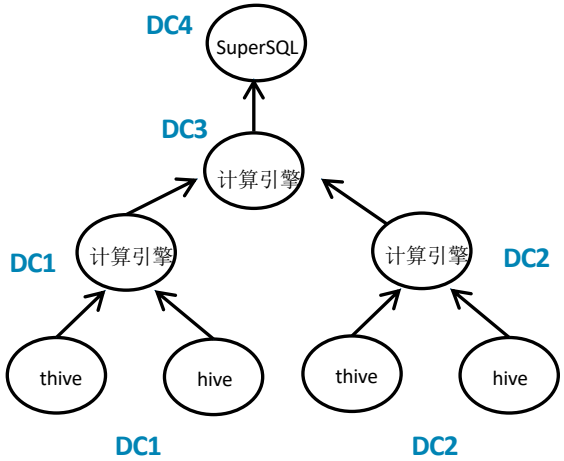
跨DC CBO：最优引擎挑选

SuperSQL

最优（单）计算引擎挑选



子查询切分，多引擎协同



① 对每个数据源下推SQL，基于统计信息估算其输出行数与字节数
② 对每个计算引擎，计算并行接收所有下推SQL结果的网络代价
③ 所有计算引擎CPU代价认为等同，基于网络代价挑选最优引擎执行该查询，这里还需考虑SuperSQL Server读/写最终查询结果的网络代价

- 数据源先与本地计算引擎匹配（基于DS前缀名），将计算分到两级的多个计算引擎
- 二级计算引擎看做一级引擎的特殊数据源
- 根据若干因子（如数据量、SQL模板、用户响应时间或稳定性要求等）挑选合适类型的计算引擎

08

性能评测



测试环境

- 集群规模：6台服务器
- 硬件：CPU 48核 M10，128GB 内存，3TB 硬盘
- OS：TLinux 2.2 64bit Version 2.2 20190320
- JDK：RedHat OpenJDK "1.8.0_131”
- 大数据套件：TBDS 5.0.0.0，基本参数调优
  - Spark 2.3.2 – 计算引擎
  - Hive 2.2.0 – 数据源（TEZ）
  - PostgreSQL 9.2.14 – 数据源
  - Hadoop 2.7.2
- 数据集：TPC-DS 1G/100G/1T
- 查询负载：TPC-DS 99条SQL
- 对比基线：社区SparkSQL JDBC

TPC-DS性能测试结果

在相同数据源情况下，数据量越大，SuperSQL的性能优势越明显

TPC-DS数据集规模	数据源	TPC-DS 查询数量百分比		
		SuperSql / Spark Jdbc 时间比值 <= 0.2	SuperSql / Spark Jdbc 时间比值 0.2~0.5	SuperSql / Spark Jdbc 时间比值 0.5~1.0
100GB	Hive	98.06%	0.97%	0.97%
	Hive + PG 混合	85.44%	7.77%	6.80%

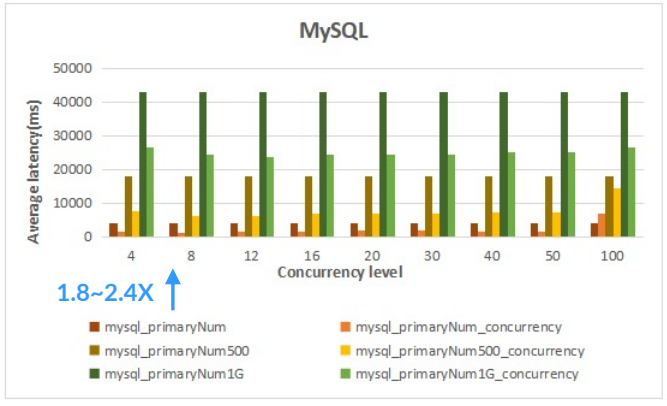
所有99条SQL的执行时间（量级几十秒到几十分钟）均值，单源Hive **26倍**提升，跨源Hive+PG **5倍**提升

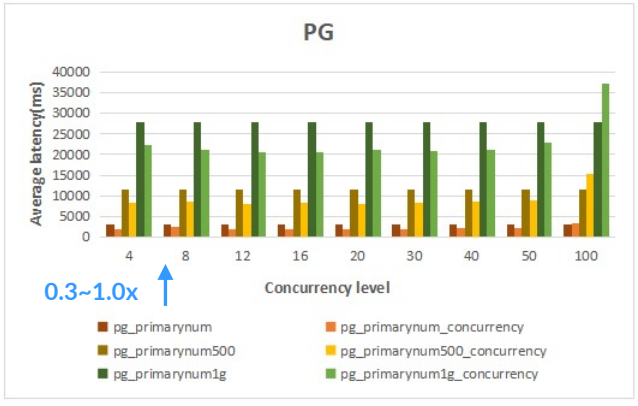
TPC-DS数据集规模	数据源	TPC-DS 查询数量百分比	
		SuperSql / Spark Jdbc 时间比值 <= 0.5	SuperSql / Spark Jdbc 时间比值 0.5~1.0
1GB	Hive	43.69%	56.31%
	PostgreSQL	81.55%	18.45%
	Hive + PG 混合	38.83%	61.17%

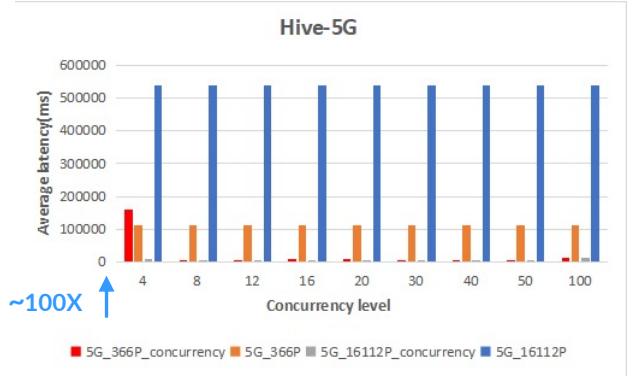
所有99条SQL的执行时间（量级几秒到十几秒）均值，单源Hive **86%**提升，单源PG **60%**提升，跨源Hive + PG **83%**提升

并发子查询性能测试结果

并发度=8~16时性能最佳

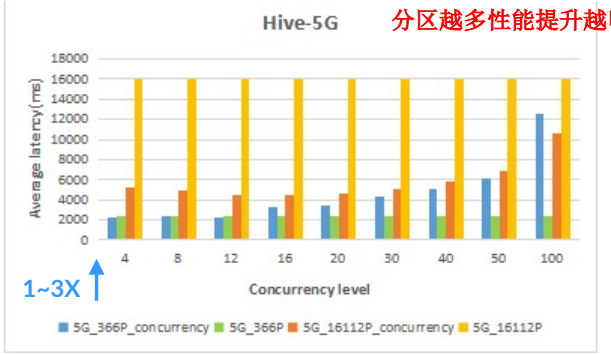






hive.fetch.task.conversion.threshold = 1G

分区越多性能提升越明显



hive.fetch.task.conversion.threshold = -1

09
系统架构

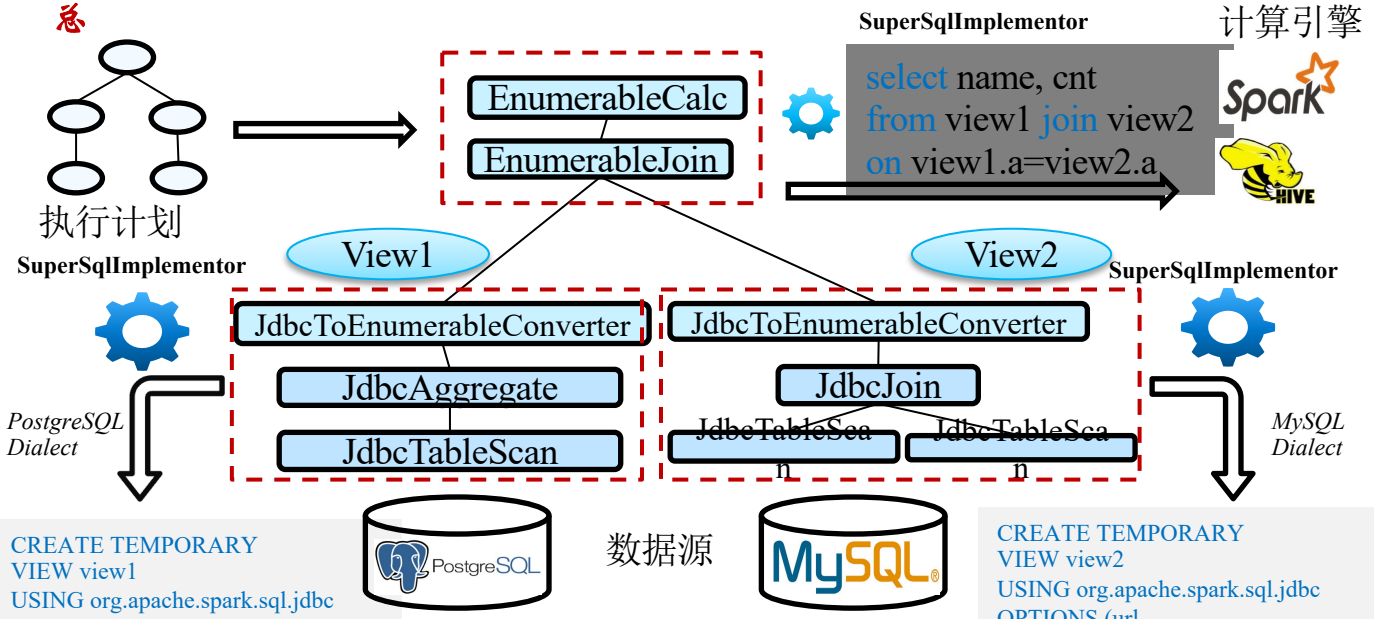


SuperSQL分布式计算引擎需求

- **快速构建**：不重复“造轮子”，尽量复用通用开源软件
- **轻量级解耦**：不强依赖特定框架，不做或少做侵入性修改
- **场景适配**：根据SQL特征，智能挑选主流执行引擎

计算引擎处理流程：数据源下推子计划 ->临时表映射

SuperSQL CBO-> 通知计算引擎 (二次优化) -> 数据源执行 -> 计算引擎汇总




```
CREATE TEMPORARY
VIEW view1
USING org.apache.spark.sql.jdbc
OPTIONS (url
'jdbc:postgresql://host1:port1/db1',
  dbtable '(select a, count(*) as cnt
from t1 group by a) t',
user "******", password "******");
```

SuperSQL外接执行引擎要求：

1. 提供JDBC Client/Server
2. 能够通过JDBC/HDFS/定制Connector连接到各类SuperSQL数据源
3. 支持动态增加/删除数据源或数据源表

```
CREATE TEMPORARY
VIEW view2
USING org.apache.spark.sql.jdbc
OPTIONS (url
"jdbc:mysql://host2:port2/db2",
  dbtable '(select a,  name from t2
join t3 on t2.id = t3.id) t',
user "******", password "******");
```

SuperSQL未来规划

- HBO
- 元数据存储重构迁移
- 基于多代价因子的优化
- 对接MPP计算引擎
- 最优执行引擎的智能选择框架

Thank you

非常感谢您的观看

DataFunCon 2021

